RUANG UMKM

Demo dan Bedah Kode Modul Pembeli Ruang UMKM

By: Aldrin Faza Huda
Leader of Project Mini Startup Ruang UMKM

ruang_umkm.py

```python
import json
import logging
from datetime import datetime

# ============================
#  RUANG UMKM - Modul Pembeli
# ============================

class AplikasiPembeli:
    def __init__(self):
        self.katalog = Katalog.dari_data_menu()
        self.riwayat = []
```

VARIABLE & DATA TYPES

</>

Bagian ini mendefinisikan semua konstanta global yang dipakai di seluruh program. MAX_PEMAKAIAN = 5 dan MAX_PERCOBAAN = 5 adalah batas maksimal pemakaian voucher dan percobaan input kode voucher. DISKON_VOUCHER_PERSEN = 10 adalah persentase diskon tetap saat voucher dipakai. TIER_DISKON adalah list of tuple [(500000, 10), (300000, 7.5), (200000, 5)] yang mendefinisikan diskon tambahan berjenjang — diurutkan dari tier terbesar agar saat pengecekan bisa berhenti di tier tertinggi pertama yang terpenuhi. STATUS_PESANAN, METODE_PEMBAYARAN, DAFTAR_BANK, dan DAFTAR_EWALLET adalah list string yang dipakai sebagai pilihan menu. DATA_MENU adalah list of tuple berisi 10 produk awal toko, dan DATA_VOUCHER adalah dict berisi 3 kode voucher dengan counter pemakaian awal 0. Semua konstanta ini sengaja diletakkan di atas agar mudah diubah tanpa harus masuk ke dalam logika fungsi.

config.py

```python
# ---------- VARIABLES & DATA TYPES ----------
MAX_PEMAKAIAN = 5          # batas maksimal 1 kode voucher dipakai
MAX_PERCOBAAN = 5          # batas salah ketik kode voucher
DISKON_VOUCHER_PERSEN = 10  # diskon voucher dari total belanja (%)

# Diskon tambahan berdasarkan total belanja: (minimal belanja, persen)
# Diurutkan dari yang terbesar agar pengecekan berhenti di tier tertinggi.
TIER_DISKON = [(500000, 10), (300000, 7.5), (200000, 5)]
BATAS_DISKON_TAMBAHAN = TIER_DISKON[-1][0]

STATUS_PESANAN = ["diproses", "dikirim", "selesai"]
METODE_PEMBAYARAN = ["Tunai (COD)", "Transfer Bank", "E-Wallet", "QRIS"]
DAFTAR_BANK = ["BRI", "BNI", "Mandiri", "BCA", "BSI", "CIMB Niaga",
"Permata"]
DAFTAR_EWALLET = ["ShopeePay", "DANA", "GoPay", "OVO", "LinkAja"]
NAMA_TOKO = "Warung Makan Sederhana"
FORMAT_TANGGAL = "%d-%m-%Y %H:%M"
```

KONFIGURASI FILE & DEBUGGING

config.py

```python
# --------- KONFIGURASI FILE & DEBUGGING ----------
FILE_DATA = "data_ruang_umkm.json" # stok, voucher, riwayat
FILE_LOG = "debug.log"             # catatan debugging
FOLDER_STRUK = "struk"             # satu file .txt per transaksi
FILE_LAPORAN = "laporan_riwayat.txt" # hasil ekspor laporan
DEBUG_MODE = False                 # True = log detail (level DEBUG)

logging.basicConfig(
    filename=FILE_LOG,
    level=logging.DEBUG if DEBUG_MODE else logging.INFO,
    format="%(asctime)s [%(levelname)]%(funcName)s: %(message)s",
    encoding="utf-8",
)
log = logging.getLogger("ruang_umkm")
```

config.py

```python
# Data contoh menu Warung Makan Sederhana:
# (id, nama, kategori, harga, stok)
DATA_MENU = [
    (1, "Nasi Putih (1 porsi)", "Nasi", 5000, 50),
    (2, "Telur Dadar / Ceplok", "Lauk", 5000, 30),
    (3, "Orek Tempe", "Lauk", 3000, 25),
    (4, "Ayam Goreng", "Lauk", 10000, 30),
    (5, "Ikan Tongkol Balado", "Lauk", 8000, 20),
    (6, "Sayur Daun Singkong", "Sayur", 3000, 20),
    (7, "Perkedel Kentang", "Lauk", 3000, 25),
    (8, "Tumis Usus", "Lauk", 4000, 20),
    (9, "Sayur Sop", "Sayur", 4000, 20),
    (10, "Es Teh Manis", "Minuman", 4000, 40),
]

# Daftar kode voucher valid beserta jumlah pemakaian awal
DATA_VOUCHER = {"48213976": 0, "70562184": 0, "93748215": 0}
```

Bagian ini mengatur nama-nama file I/O dan sistem logging. FILE_DATA menunjuk ke data_ruang_umkm.json sebagai database utama yang menyimpan stok, voucher, dan riwayat transaksi. FILE_LOG adalah debug.log untuk mencatat semua aktivitas program. FOLDER_STRUK adalah folder struk/ tempat setiap struk transaksi disimpan sebagai file .txt terpisah. DEBUG_MODE = False mengontrol level logging — jika True, semua log level DEBUG ikut dicatat; jika False, hanya level INFO ke atas. Konfigurasi logging.basicConfig() menyetel format log dengan timestamp, level, dan nama fungsi sumber log. Logger log ini kemudian dipakai di seluruh program dengan log.debug(), log.info(), log.warning(), dan log.error() untuk memudahkan debugging tanpa mengganggu output di terminal.

FUNGSI BANTU INPUT

input_helper.py

```python
# --------- FUNGSI BANTU INPUT ---------
def input_angka(pesan, tipe=int, pesan_min="Nilai harus
lebih dari 0. Coba lagi."):

    """Looping sampai user memasukkan angka valid (> 0)."""
    while True:
        try:
            nilai = tipe(input(pesan))
            if nilai <= 0:
                print(pesan_min)
                continue

            return nilai
    except ValueError:
        log.debug("Input angka tidak valid pada prompt: %s",
        pesan)
        print("Input tidak valid, masukkan angka.")
```

input_helper.py

```python
def input_pilihan(pesan, minimal, maksimal):
    """Looping sampai user memilih angka dalam rentang minimal..maksimal."""
    while True:
        try:
            nilai = int(input(pesan))
            if minimal <= nilai <= maksimal:
                return nilai
            print(f"Pilihan harus antara {minimal} dan {maksimal}.")
        except ValueError:
            log.debug("Input pilihan tidak valid pada prompt: %s", pesan)
            print("Input tidak valid, masukkan angka.")

def input_yn(pesan):
    """Looping sampai user memasukkan 'y' atau 'n'. Mengembalikan True/False."""
    while True:
        jawab = input(pesan).strip().lower()
        if jawab in ("y", "n"):
            return jawab == "y"
        print("Input tidak valid! Harap masukkan 'y' atau 'n'.")
```

input_helper.py

```python
def rp(nilai):
    return "Rp" + format(nilai, ",")

def bulatkan(nilai):
    """Pembulatan ke rupiah terdekat (0.5 dibulatkan ke atas)."""
    return int(nilai + 0.5)

def pilih_dari_daftar(judul, daftar):
    print(f"\n{judul}")
    for i, nama in enumerate(daftar, start=1):
        print(f"  {i}. {nama}")
    pilihan = input_pilihan("Pilihan: ", 1, len(daftar))
    return daftar[pilihan - 1]

def pilih_metode_pembayaran():
    """Metode utama, lalu (jika perlu) pilih bank / e-wallet yang dipakai."""
    metode = pilih_dari_daftar("Pilih metode pembayaran:", METODE_PEMBAYARAN)

    if metode == "Transfer Bank":
        bank = pilih_dari_daftar("Pilih bank:", DAFTAR_BANK)
        return f"Transfer Bank - {bank}"
    if metode == "E-Wallet":
        ewallet = pilih_dari_daftar("Pilih e-wallet:", DAFTAR_EWALLET)
        return f"E-Wallet - {ewallet}"
    return metode  # Tunai (COD) atau QRIS
```

Bagian ini berisi kumpulan fungsi utilitas input yang memastikan user tidak bisa memasukkan data yang tidak valid. input_angka() melakukan looping hingga user memasukkan angka bertipe int atau float yang lebih dari 0 — cocok untuk input jumlah barang. input_pilihan() memastikan input berupa angka dalam rentang minimal hingga maksimal — dipakai untuk memilih dari menu bernomor. input_yn() menerima hanya 'y' atau 'n' dan mengembalikan True/False — dipakai untuk semua pertanyaan konfirmasi. Fungsi rp() memformat angka integer menjadi string Rupiah seperti "Rp10,000". bulatkan() melakukan pembulatan ke rupiah terdekat dengan logika int(nilai + 0.5). pilih_dari_daftar() menampilkan list bernomor dan mengembalikan item yang dipilih. pilih_metode_pembayaran() adalah fungsi khusus yang menggabungkan pilihan metode utama (Tunai/Transfer/E-Wallet/QRIS) dengan sub-pilihan bank atau e-wallet jika diperlukan.

ALGORITMA SORT & SEARCH

sort_search.py

```python
# --------- ALGORITMA SORT & SEARCH ----------
def urutkan_data(data, kunci, menurun=False):
    """SORT: insertion sort (stabil). Mengembalikan list baru yang sudah
       terurut berdasarkan fungsi `kunci`; data asli tidak diubah.
    """
    hasil = list(data)
    for i in range(1, len(hasil)):
        sisip = hasil[i]
        nilai = kunci(sisip)
        j = i - 1
        while j >= 0 and (kunci(hasil[j]) < nilai if menurun
            hasil[j + 1]  else kunci(hasil[j]) > nilai):
            j -= 1
        hasil[j + 1] = sisip
    log.debug("Sort selesai: %d data, menurun=%s", len(hasil), menurun)
    return hasil

def binary_search_id(daftar, id_cari):
    """SEARCH: binary search produk berdasarkan id.
    Syarat: `daftar` sudah terurut menaik berdasarkan id."""
    kiri, kanan = 0, len(daftar) - 1
    while kiri <= kanan:
        tengah = (tengah].id == id_cari: return daftar[tengah]
        if daftar[tengah].id < id_cari: kanan = tengah - 1
    return None
```

Bagian ini mengimplementasikan dua algoritma secara manual tanpa library bawaan Python. urutkan_data() menggunakan Insertion Sort yang bersifat stabil — data diurutkan berdasarkan fungsi kunci yang dilewatkan sebagai parameter, dan parameter menurun mengontrol arah urutan naik/turun. Fungsi ini mengembalikan list baru sehingga data asli tidak berubah, dan dipakai di banyak tempat: mengurutkan katalog produk, riwayat transaksi, maupun daftar nama toko saat ulasan. binary_search_id() menggunakan Binary Search untuk mencari produk berdasarkan ID secara efisien dengan syarat list sudah terurut menaik by ID — ini dijamin karena Katalog selalu menyimpan produk terurut. Selain itu, ada Linear Search yang diimplementasikan secara inline di beberapa tempat — seperti di cari_kata_kunci() pada Katalog dan _cari_riwayat() pada AplikasiPembeli — menggunakan list comprehension untuk menyaring data berdasarkan kata kunci.

OOP: KATALOG & PRODUK

ruang_umkm.py — 2 / 3 • class Katalog — awal

```python
class Katalog:
    # atribut yang boleh dipakai untuk pengurutan: label -> fungsi kunci
    KUNCI_URUT = {
        "Nama": lambda p: p.nama.lower(),
        "Harga": lambda p: p.harga,
        "Stok": lambda p: p.stok,
        "Kategori": lambda p: p.kategori.lower(),
    }

    def __init__(self, daftar_produk):
        # selalu disimpan terurut berdasarkan id agar binary search valid
        self.daftar = urutkan_data(daftar_produk, lambda p: p.id)

    @classmethod
    def dari_data_menu(cls):
        return cls([Produk(i, nama, NAMA_TOKO, kat, harga, stok)
                    for i, nama, kat, harga, stok in DATA_MENU])

    def cari_id(self, id_produk):
        """Search: binary search berdasarkan id."""
        return binary_search_id(self.daftar, id_produk)
```

ruang_umkm.py — 2 / 3 • class Katalog — awal

```python
class Katalog:
    # atribut yang boleh dipakai untuk pengurutan: label -> fungsi kunci
    KUNCI_URUT = {
        "Nama": lambda p: p.nama.lower(),
        "Harga": lambda p: p.harga,
        "Stok": lambda p: p.stok,
        "Kategori": lambda p: p.kategori.lower(),
    }

    def __init__(self, daftar_produk):
        # selalu disimpan terurut berdasarkan id agar binary search valid
        self.daftar = urutkan_data(daftar_produk, lambda p: p.id)

    @classmethod
    def dari_data_menu(cls):
        return cls([Produk(i, nama, NAMA_TOKO, kat, harga, stok)
                    for i, nama, kat, harga, stok in DATA_MENU])

    def cari_id(self, id_produk):
        """Search: binary search berdasarkan id."""
        return binary_search_id(self.daftar, id_produk)
```

ruang_umkm.py — 1 / 3 • class Produk

```python
# ---------- OOP: PRODUK & KATALOG ----------
class Produk:
    def __init__(self, id, nama, toko, kategori, harga, stok):
        self.id = id
        self.nama = nama
        self.toko = toko
        self.kategori = kategori
        self.harga = harga
        self.stok = stok

    def kurangi_stok(self, qty):
        if qty > self.stok:
            raise ValueError(
                f"Stok '{self.nama}' tidak cukup (stok {self.stok}, minta {qty})")
        self.stok -= qty
        log.debug("Stok '%s' berkurang %d -> sisa %d", self.nama, qty, self.stok)
```

Produk adalah class data sederhana yang menyimpan atribut id, nama, toko, kategori, harga, dan stok. Method kurangi_stok(qty) memvalidasi apakah stok cukup sebelum mengurangi, dan melempar ValueError jika tidak — ini adalah satu-satunya titik pengurangan stok sehingga tidak bisa terjadi stok negatif dari luar. Katalog adalah class yang mengelola seluruh koleksi produk dan selalu menyimpan daftar dalam urutan menaik by ID agar binary search valid. Class ini punya atribut class KUNCI_URUT berupa dict yang memetakan label urutan ("Nama", "Harga", "Stok", "Kategori") ke lambda function yang menjadi kunci sort. Method dari_data_menu() adalah class method factory yang membangun katalog dari DATA_MENU. cari_id() memanggil binary search, cari_kata_kunci() melakukan linear search pada nama/toko/kategori, urutkan() mengembalikan salinan terurut, tampilkan() mencetak tabel ke terminal, dan verifikasi_stok() adalah method debugging yang memastikan tidak ada stok negatif setelah transaksi.

OOP: KERANJANG

ruang_umkm.py — 1 / 2 • class Item...

```python
# ---------- OOP: KERANJANG ----------
class ItemKeranjang:
    def __init__(self, produk, qty):
        self.produk = produk
        self.qty = qty

    @property
    def id(self):
        return self.produk.id

    @property
    def nama(self):
        return self.produk.nama

    @property
    def toko(self):
        return self.produk.toko

    @property
    def harga(self):
        return self.produk.harga

    @property
    def total(self):
        return self.produk.harga * self.qty

    def to_dict(self):
        return {"id": self.id, "nama": self.nama, "toko": self.toko,
                "harga": self.harga, "qty": self.qty, "total": self.total}
```

ruang_umkm.py — 2 / 2 • class Keranjang

```python
class Keranjang:
    def __init__(self):
        self.items = []

    def kosong(self):
        return len(self.items) == 0

    def jumlah_produk(self, id_produk):
        for item in self.items:
            if item.id == id_produk:
                return item.qty
        return 0

    def tambah(self, produk, qty):
        # Produk yang sama digabung, bukan dibuat baris baru
        for item in self.items:
            if item.id == produk.id:
                item.qty += qty
                return
        self.items.append(ItemKeranjang(produk, qty))

    def ubah_jumlah(self, index, qty_baru):
        self.items[index].qty = qty_baru

    def hapus(self, index):
        return self.items.pop(index)

    @property
    def subtotal(self):
        return sum(item.total for item in self.items)

    @property
    def total_qty(self):
        return sum(item.qty for item in self.items)

    def tampilkan(self):
        """Menampilkan daftar belanja sementara (mirip struk)."""
        print("\n---------- DAFTAR BELANJA SAAT INI ----------")
        print(f"{'No':<3}{'Nama Barang':<24}{'Qty':>4}{'Harga':>9}{'Total':>10}")
        for i, item in enumerate(self.items, start=1):
            print(f"{i:<3}{item.nama[:23]:<24}{item.qty:>4}"
                  f"{item.harga:>9,}{item.total:>10,}")
        print("---------------------------------------------\n")
```

ItemKeranjang adalah class kecil yang merepresentasikan satu baris di keranjang — menyimpan referensi ke objek Produk dan qty yang dipesan. Property total menghitung harga × qty secara dinamis, dan to_dict() mengonversi item ke dictionary untuk disimpan ke JSON. Keranjang adalah class utama yang menyimpan list items bertipe ItemKeranjang. Method tambah(produk, qty) memiliki logika penggabungan — jika produk yang sama sudah ada di keranjang, qty-nya dijumlahkan bukan dibuat baris baru. ubah_jumlah() dan hapus() memodifikasi item berdasarkan index. Property subtotal menjumlahkan seluruh item.total, property kosong() mengecek apakah keranjang masih kosong, dan tampilkan() mencetak tabel daftar belanja ke terminal. Keranjang baru dibuat di setiap transaksi dan tidak persisten antar-sesi — hanya Transaksi yang disimpan ke file.

OOP: VOUCHER

ruang_umkm.py — 1 / 2 • VoucherManager — awal

```python
# ---------- OOP: VOUCHER ----------
class VoucherManager:
    def __init__(self, pemakaian=None):
        # dict kode -> jumlah pemakaian (tracking count)
        self.pemakaian = dict(DATA_VOUCHER)
        if pemakaian:
            for kode, jumlah in pemakaian.items():
                if kode in self.pemakaian:
                    self.pemakaian[kode] = int(jumlah)

    def sisa_pakai(self, kode):
        return MAX_PEMAKAIAN - self.pemakaian[kode]

    def pakai(self, kode):
        self.pemakaian[kode] += 1
        log.info("Voucher %s dipakai (total %d x)", kode, self.pemakaian[kode])

    def validasi_interaktif(self):
        """
        - Setiap kode maksimal bisa dipakai 5x (MAX_PEMAKAIAN) sebelum "hangus".
        - Kode yang salah ketik ATAU sudah hangus sama-sama dihitung sebagai
          percobaan gagal, dan user diberi kesempatan input ulang sampai
          MAX_PERCOBAAN kali.
        Mengembalikan kode voucher yang valid, atau None jika tidak memakai
        voucher. (Counter pemakaian baru bertambah setelah pesanan dikonfirmasi.)
        """
        percobaan = 0
        while percobaan < MAX_PERCOBAAN:
            kode_input = input(
                "Masukkan kode voucher (kosongkan jika tidak punya): "
            ).strip()

            if kode_input == "":
                print("Anda tidak memasukkan kode voucher. "
                      "Anda membayar dengan harga normal.")
                return None

            percobaan += 1
```

ruang_umkm.py — 2 / 2 • validasi + status

```python
            sisa_percobaan = MAX_PERCOBAAN - percobaan

            if kode_input not in self.pemakaian:
                log.warning("Kode voucher tidak dikenal: %s", kode_input)
                print("Kode voucher tidak valid.", end=" ")
            elif self.pemakaian[kode_input] >= MAX_PEMAKAIAN:
                log.warning("Kode voucher hangus: %s", kode_input)
                print(f"Kode voucher '{kode_input}' sudah hangus "
                      f"(sudah dipakai {MAX_PEMAKAIAN}x).", end=" ")
            else:
                print("Kode valid! Voucher berhasil diterapkan.")
                return kode_input

            if sisa_percobaan > 0:
                print(f"Sisa kesempatan input: {sisa_percobaan}x")
            else:
                print("Kesempatan input kode voucher sudah habis. "
                      "Anda membayar dengan harga normal.")
        return None

    def tampilkan_status(self, kode_dipakai):
        print("===== STATUS VOUCHER ANDA =====")
        if kode_dipakai:
            print(f"Kode voucher digunakan   : {kode_dipakai}")
            print(f"Sisa penggunaan kode ini : {self.sisa_pakai(kode_dipakai)}x lagi")
        else:
            print("Tidak ada voucher yang digunakan pada transaksi ini.")
        print("================================\n")
```

VoucherManager mengelola semua logika validasi dan tracking pemakaian voucher. Konstruktornya menerima parameter pemakaian opsional untuk memuat data pemakaian yang sudah tersimpan dari sesi sebelumnya — jika tidak ada, dipakai DATA_VOUCHER sebagai default. Method validasi_interaktif() adalah method utama yang mengimplementasikan alur voucher secara lengkap: user diberi maks MAX_PERCOBAAN (5) kali untuk memasukkan kode, dan setiap kode yang salah ketik maupun yang sudah hangus sama-sama dihitung sebagai satu percobaan gagal. Jika user mengosongkan input, method langsung mengembalikan None tanpa menghitung percobaan. Jika kode valid, method mengembalikan kode tersebut, namun counter pemakaian belum ditambah di sini — penambahan dilakukan hanya setelah pesanan benar-benar dikonfirmasi bayar, sehingga pembatalan pesanan tidak membuang jatah. pakai() menambah counter, sisa_pakai() mengembalikan sisa pemakaian, dan tampilkan_status() mencetak info voucher setelah checkout.

OOP: TRANSAKSI

Transaksi adalah class yang merepresentasikan satu transaksi lengkap dengan semua atributnya: no_transaksi, tanggal, pembeli, lokasi, metode, items (list of dict), voucher, angka-angka keuangan, chat, ulasan, dan status. Static method hitung_pembayaran(subtotal, pakai_voucher) adalah inti logika keuangan — menghitung diskon voucher 10% dari subtotal, lalu diskon tambahan dari total setelah diskon voucher berdasarkan TIER_DISKON, dan mengembalikan dict lengkap berisi semua nilai keuangan. Method verifikasi() adalah fungsi debugging yang memeriksa konsistensi angka — apakah subtotal cocok dengan jumlah total item, apakah total_bayar = subtotal − diskon, apakah total_hemat benar — dan melempar DataTidakKonsisten jika ada yang janggal. buat_struk() menghasilkan string teks struk belanja yang terformat rapi. to_dict() dan from_dict() dipakai untuk serialisasi ke/dari JSON.

ruang_umkm.py — 1 / 4 • __init__

```python
# ---------- OOP: TRANSAKSI ----------
class DataTidakKonsisten(Exception):
    """Dilempar saat verifikasi data (debugging) menemukan nilai yang janggal."""


class Transaksi:
    def __init__(self, no_transaksi, tanggal, pembeli, lokasi, metode, items,
                 total_qty, voucher, subtotal, nilai_diskon, potongan_khusus,
                 nilai_diskon_khusus, total_bayar, total_hemat,
                 chat=None, ulasan=None, status="diproses"):
        self.no_transaksi = no_transaksi
        self.tanggal = tanggal
        self.pembeli = pembeli
        self.lokasi = lokasi
        self.metode = metode
        self.items = items              # list of dict (hasil to_dict item)
        self.total_qty = total_qty
        self.voucher = voucher
        self.subtotal = subtotal
        self.nilai_diskon = nilai_diskon
        self.potongan_khusus = potongan_khusus
        self.nilai_diskon_khusus = nilai_diskon_khusus
        self.total_bayar = total_bayar
        self.total_hemat = total_hemat
        self.chat = chat if chat is not None else []
        self.ulasan = ulasan if ulasan is not None else []
        self.status = status
```

ruang_umkm.py — 2 / 4 • hitung_pembayaran

```python
    # --- perhitungan (OPERATORS & EXPRESSIONS) ---
    @staticmethod
    def hitung_pembayaran(subtotal, pakai_voucher):
        """
        Diskon voucher 10% dari subtotal; diskon tambahan (5% / 7,5% / 10%)
        ditentukan dari subtotal dan dihitung dari total setelah diskon voucher.
        """
        nilai_diskon = 0
        nilai_diskon_khusus = 0
        potongan_khusus = 0

        if pakai_voucher:
            nilai_diskon = bulatkan(subtotal * DISKON_VOUCHER_PERSEN / 100)
            total_bayar = subtotal - nilai_diskon

            for batas, persen in TIER_DISKON:
                if subtotal >= batas:
                    potongan_khusus = persen
                    break

            nilai_diskon_khusus = bulatkan(total_bayar * potongan_khusus / 100)
            total_bayar -= nilai_diskon_khusus
        else:
            total_bayar = subtotal

        return {
            "subtotal": subtotal,
            "nilai_diskon": nilai_diskon,
            "potongan_khusus": potongan_khusus,
            "nilai_diskon_khusus": nilai_diskon_khusus,
            "total_bayar": total_bayar,
            "total_hemat": subtotal - total_bayar,
        }
```

ruang_umkm.py — 3 / 4 • verifikasi + struk awal

```python
    # --- debugging ---
    def verifikasi(self):
        """Memeriksa konsistensi angka transaksi. Melempar DataTidakKonsisten."""
        if self.subtotal != sum(i["total"] for i in self.items):
            raise DataTidakKonsisten(f"{self.no_transaksi}: subtotal tidak sama "
                                     f"dengan jumlah total item")
        if self.total_qty != sum(i["qty"] for i in self.items):
            raise DataTidakKonsisten(f"{self.no_transaksi}: total_qty salah")
        harapan = self.subtotal - self.nilai_diskon - self.nilai_diskon_khusus
        if self.total_bayar != harapan:
            raise DataTidakKonsisten(
                f"{self.no_transaksi}: total_bayar {self.total_bayar} "
                f"seharusnya {harapan}")
        if self.total_bayar < 0:
            raise DataTidakKonsisten(f"{self.no_transaksi}: total_bayar negatif")
        if self.total_hemat != self.subtotal - self.total_bayar:
            raise DataTidakKonsisten(f"{self.no_transaksi}: total_hemat salah")

    # --- output struk (dikembalikan sebagai teks agar bisa dicetak & disimpan) ---
    def buat_struk(self):
        garis = "=" * 50
        b = [garis,
             "STRUK BELANJA".center(50),
             NAMA_TOKO.center(50),
             garis,
             f"No. Transaksi : {self.no_transaksi}",
             f"Tanggal       : {self.tanggal}",
             f"Nama Pembeli  : {self.pembeli}",
             f"Lokasi        : {self.lokasi}",
             f"Pembayaran    : {self.metode}",
             "-" * 50,
             f"{'Nama Barang':<24}{'Qty':>4}{'Harga':>10}{'Total':>12}",
             "-" * 50]

        for item in self.items:
            b.append(f"{item['nama'][:23]:<24}{item['qty']:>4}"
                     f"{item['harga']:>10,}{item['total']:>12,}")
```

ruang_umkm.py — 4 / 4 • struk lanjutan + to_dict

```python
        b.append("-" * 50)
        b.append(f"{'Jumlah Item':<30}{self.total_qty:>20}")
        b.append(f"{'Subtotal':<30}{rp(self.subtotal):>20}")

        if self.voucher:
            b.append(f"{'VC Diskon Voucher ' + str(DISKON_VOUCHER_PERSEN) + '%':<30}"
                     f"{'(' + format(self.nilai_diskon, ',') + ')':>20}")
            if self.potongan_khusus > 0:
                label = ('VC Diskon Tambahan '
                         + format(self.potongan_khusus, '.1f') + '%')
                b.append(f"{label:<30}"
                         f"{'(' + format(self.nilai_diskon_khusus, ',') + ')':>20}")

        b.append("-" * 50)
        b.append(f"{'TOTAL BAYAR':<30}{rp(self.total_bayar):>20}")
        b.append(f"{'ANDA HEMAT':<30}{rp(self.total_hemat):>20}")
        b.append(garis)

        # Pesan keterangan voucher
        if self.voucher and self.potongan_khusus > 0:
            b.append(f"Selamat! Voucher anda berhasil digunakan, kamu mendapatkan "
                     f"diskon tambahan {self.potongan_khusus:.1f}%")
        elif self.voucher:
            b.append(f"Dengan voucher, belanja minimal {rp(BATAS_DISKON_TAMBAHAN)} "
                     f"agar dapat diskon tambahan")
        else:
            b.append("Anda membayar dengan harga normal (tanpa diskon apapun)")
        b.append("       Terima kasih telah berbelanja!")
        return "\n".join(b)

    # --- serialisasi untuk file JSON ---
    def to_dict(self):
        return dict(self.__dict__)

    @classmethod
    def from_dict(cls, data):
        return cls(**data)
```

ruang_umkm.py 1 / 2 • muat

```python
# ---------- OOP: PENYIMPANAN (I/O FILE) ----------
class PenyimpananData:
    """Semua operasi baca/tulis file dikumpulkan di sini."""

    def __init__(self, file_data=FILE_DATA):
        self.file_data = file_data

    def muat(self):
        """Membaca file JSON. Mengembalikan dict, atau None jika tidak ada/rusak."""
        if not os.path.exists(self.file_data):
            log.info("File data belum ada, memakai data awal.")
            return None
        try:
            with open(self.file_data, "r", encoding="utf-8") as f:
                data = json.load(f)
            log.info("Data dimuat dari %s", self.file_data)
            return data
        except (json.JSONDecodeError, OSError) as err:
            log.error("Gagal membaca %s: %s", self.file_data, err)
            print(f"Peringatan: file data tidak bisa dibaca ({err}). "
                  f"Program memakai data awal.")
            try:
                os.replace(self.file_data, self.file_data + ".rusak")
                log.info("File rusak dipindah ke %s.rusak", self.file_data)
            except OSError:
                pass
            return None
```

ruang_umkm.py 2 / 2 • simpan + simpan_teks

```python
    def simpan(self, data):
        """Menulis JSON secara aman (tulis ke file sementara lalu ganti)."""
        sementara = self.file_data + ".tmp"
        try:
            with open(sementara, "w", encoding="utf-8") as f:
                json.dump(data, f, indent=2, ensure_ascii=False)
            os.replace(sementara, self.file_data)
            log.debug("Data disimpan ke %s", self.file_data)
            return True
        except OSError as err:
            log.exception("Gagal menyimpan %s", self.file_data)
            print(f"Peringatan: data gagal disimpan ({err}).")
            return False

    def simpan_teks(self, path, teks):
        """Menulis file teks (struk / laporan)."""
        try:
            folder = os.path.dirname(path)
            if folder:
                os.makedirs(folder, exist_ok=True)
            with open(path, "w", encoding="utf-8") as f:
                f.write(teks + "\n")
            log.info("File teks disimpan: %s", path)
            return True
        except OSError as err:
            log.exception("Gagal menulis %s", path)
            print(f"Peringatan: gagal menulis file {path} ({err}).")
            return False

    def simpan_struk(self, trx):
        return self.simpan_teks(
            os.path.join(FOLDER_STRUK, f"{trx.no_transaksi}.txt"),
            trx.buat_struk())
```

OOP: PENYIMPANAN

PenyimpananData adalah class yang mengenkapsulasi semua operasi baca/tulis file sehingga logika I/O tidak tersebar di seluruh kode. Method muat() membaca file JSON dan mengembalikan dict data, atau None jika file belum ada atau rusak — jika rusak, file tersebut di-rename menjadi .rusak agar tidak terus-menerus gagal dibaca. Method simpan() menulis JSON dengan teknik atomic write — data pertama ditulis ke file sementara .tmp, baru kemudian di-replace ke nama file asli dengan os.replace(), sehingga jika program crash di tengah penulisan, file data lama tetap utuh dan tidak korup. simpan_teks() menulis file teks biasa (struk atau laporan) dengan membuat folder jika belum ada. simpan_struk() adalah wrapper khusus yang memanggil simpan_teks() dengan path struk/{no_transaksi}.txt.

OOP: APLIKASI PEMBELI

AplikasiPembeli adalah class orkestrator yang menyatukan semua komponen dan menjalankan alur lengkap aplikasi. Konstruktornya menginisialisasi PenyimpananData, Katalog, VoucherManager, list riwayat, lalu memanggil _muat_data() untuk memulihkan state dari file JSON. _muat_data() memuat stok produk, data voucher, dan riwayat transaksi dari file. simpan_semua() mengumpulkan state terkini dari katalog, voucher, dan riwayat lalu menyimpannya. _buat_nomor_transaksi() membuat ID unik berformat TRX-YYYYMMDD-001 dengan counter yang dihitung dari riwayat hari yang sama. Bagian katalog (pilih_produk, menu_urutkan_katalog) menangani tampilan dan interaksi pemilihan produk dari terminal. Bagian keranjang (tambah_ke_keranjang, ubah_jumlah, hapus_barang) menangani modifikasi isi keranjang dengan validasi stok. Bagian status pesanan (pantau_status_pesanan) mensimulasikan update status diproses→dikirim→selesai oleh penjual, menangani chat, dan konfirmasi penerimaan. beri_ulasan() meminta rating 1–5 dan komentar per toko. proses_transaksi() adalah method utama yang mengorkestrasi satu transaksi dari awal sampai akhir — mulai input nama, keranjang, checkout, status, ulasan, hingga return objek Transaksi. jalankan() adalah loop utama yang memanggil proses_transaksi() berulang kali selama user ingin bertransaksi lagi.

ruang_umkm.py — 1 / 3 • __init__ + _muat_data

```python
# ---------- OOP: APLIKASI PEMBELI ----------
class AplikasiPembeli:
    def __init__(self):
        self.penyimpanan = PenyimpananData()
        self.katalog = Katalog.dari_data_menu()
        self.voucher = VoucherManager()
        self.riwayat = []          # menyimpan semua transaksi yang selesai
        self._muat_data()

    # ----- I/O FILE: muat & simpan state -----
    def _muat_data(self):
        data = self.penyimpanan.muat()
        if not data:
            return

        for kode_id, stok in data.get("stok", {}).items():
            produk = self.katalog.cari_id(int(kode_id))
            if produk is not None and int(stok) >= 0:
                produk.stok = int(stok)

        self.voucher = VoucherManager(data.get("voucher", {}))

        for d in data.get("riwayat", []):
            try:
                self.riwayat.append(Transaksi.from_dict(d))
            except (TypeError, KeyError) as err:
                log.error("Satu data riwayat dilewati (rusak): %s", err)

        print(f"Data sebelumnya dimuat: {len(self.riwayat)} transaksi "
              f"dari {self.penyimpanan.file_data}.")
```

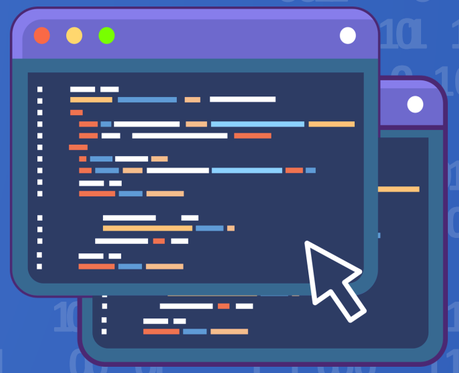
ruang_umkm.py — 2 / 3 • simpan + nomor + urutkan

```python
    def simpan_semua(self):
        data = {
            "stok": {str(p.id): p.stok for p in self.katalog.daftar},
            "voucher": self.voucher.pemakaian,
            "riwayat": [t.to_dict() for t in self.riwayat],
        }
        return self.penyimpanan.simpan(data)

    def _buat_nomor_transaksi(self):
        awalan = f"TRX-{datetime.now():%Y%m%d}-"
        urutan = sum(1 for t in self.riwayat
                     if t.no_transaksi.startswith(awalan)) + 1
        return f"{awalan}{urutan:03d}"

    # ----- KATALOG: pilih, cari, urutkan -----
    def menu_urutkan_katalog(self):
        atribut = list(Katalog.KUNCI_URUT)
        print("\nUrutkan menu berdasarkan:")
        for i, nama in enumerate(atribut, start=1):
            print(f"  {i}. {nama}")
        print("  0. Batal")
        pilih = input_pilihan("Pilihan: ", 0, len(atribut))
        if pilih == 0:
            return
        print("  1. Naik (A-Z / termurah)   2. Turun (Z-A / termahal)")
        menurun = input_pilihan("Urutan: ", 1, 2) == 2
        terurut = self.katalog.urutkan(atribut[pilih - 1], menurun)
        self.katalog.tampilkan(terurut)
```

ruang_umkm.py — 3 / 3 • pilih_produk

```python
    def pilih_produk(self):
        """
        Katalog langsung ditampilkan, user tinggal memilih nomor menu.
        Mengetik kata kunci menyaring daftar menu (linear search),
        mengetik 'urut' mengurutkan daftar menu (sort).
        Mengembalikan Produk, atau None jika dibatalkan.
        """
        print(f"\n=== KATALOG MENU - {NAMA_TOKO.upper()} ===")
        self.katalog.tampilkan()

        while True:
            masukan = input(
                "Pilih nomor menu (0 = batal, 'urut' = urutkan, "
                "atau ketik kata kunci): "
            ).strip()

            if masukan == "0":
                return None

            if masukan.isdigit():
                produk = self.katalog.cari_id(int(masukan))   # binary search
                if produk:
                    return produk
                print("Nomor menu tidak ada di daftar.")
                continue

            if masukan == "":
                self.katalog.tampilkan()
                continue

            if masukan.lower() == "urut":
                self.menu_urutkan_katalog()
                continue

            hasil = self.katalog.cari_kata_kunci(masukan)      # linear search
            if hasil:
                self.katalog.tampilkan(hasil)
            else:
                print("Menu tidak ditemukan. Coba kata kunci lain.")
```

TITIK MASUK PROGRAM



Fungsi main() adalah titik masuk program yang dipanggil oleh blok if __name__ == "__main__". Di sini AplikasiPembeli diinstansiasi, lalu app.jalankan() dipanggil untuk memulai loop utama. Fungsi ini dibungkus dengan blok try-except-finally: KeyboardInterrupt ditangkap untuk menangani penghentian paksa via Ctrl+C dengan pesan yang sopan, dan Exception generik ditangkap untuk error tak terduga yang dicatat ke log. Blok finally memastikan app.simpan_semua() selalu dipanggil di akhir apapun yang terjadi — termasuk saat crash — sehingga data terakhir tidak hilang. Pola if __name__ == "__main__" juga memastikan main() hanya dijalankan saat file ini dieksekusi langsung, bukan saat diimpor sebagai modul oleh file lain.

ruang_umkm.py

1 / 1 • main

```python
# ---------- TITIK MASUK PROGRAM ----------
def main():
    app = None
    try:
        app = AplikasiPembeli()
        app.jalankan()
    except KeyboardInterrupt:
        print("\nProgram dihentikan pengguna.")
        log.warning("Program dihentikan dengan Ctrl+C")
    except Exception:
        log.exception("Error tak terduga")
        print(f"\nTerjadi kesalahan tak terduga. Detail ada di {FILE_LOG}.")
    finally:
        if app is not None:
            app.simpan_semua()   # pastikan data terakhir tidak hilang


if __name__ == "__main__":
    main()
```

THANK YOU

www.linkedin.com/in/aldrin-faza-huda/

aldrinfazahuda@gmail.com